

# MCP Agora

Server MCP con memoria persistente cross-agente per flotte di agenti AI

*Antonio Cioffi*

*2025*

MCP Agora e' un progetto portfolio/learning che implementa un MCP Server (Model Context Protocol) con memoria persistente cross-agente per flotte di agenti AI personali. Utilizza ChromaDB per indicizzazione vettoriale, sentence-transformers per embedding semantici (modello all-MiniLM-L6-v2, 384 dimensioni), SQLite per metadati e cache persistente, e FastMCP per l'esposizione di 8 tool MCP. L'architettura e' organizzata in 7 layer (Transport, Protocol, Router, Memory, Cache, Backend Connector, Embedding) con routing semantico a backend MCP esterni (GitHub, Playwright). Il progetto conta 61 test automatici distribuiti su 10 file, zero mock, con ChromaDB e SQLite reali in directory isolate.

*Repository: <https://github.com/cioffiAI/mcp-agora>*

## 1. Introduzione e Obiettivi

---

MCP Agora e' un progetto portfolio/learning che implementa un MCP Server (Model Context Protocol) con memoria persistente cross-agente per flotte di agenti AI personali. Nasce dall'esigenza reale di risolvere un problema concreto: gli agenti AI moderni (Claude Code, Codex CLI, ChatGPT, Gemini CLI) operano in sessioni isolate, senza memoria condivisa. Ogni agente ripete ricerche, ricontesta informazioni e duplica lavoro che un altro agente ha gia' fatto.

### 1.1 Obiettivi di Prodotto

- Un agente salva conoscenza -> un altro agente la recupera (zero duplicazione)
- Una query gia' fatta -> risposta cached (zero ricalcolo)
- Un errore gia' fatto -> non si ripete (memoria delle decisioni)
- 4+ agenti condividono contesto senza configurazioni individuali

### 1.2 Obiettivi di Apprendimento

- MCP Protocol deep dive: implementazione MCP Server compliant con SDK ufficiale
- Vector search & embeddings: integrazione ChromaDB + sentence-transformers
- System design a layer: architettura modulare con 7 layer separati e interfacce astratte
- Async Python: asyncio per concorrenza su MCP transport
- SQLite avanzato: schema relazionale per metadati, provenance, analytics

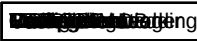
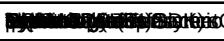
- Caching strategies: TTLCache in-memory + SQLite persistente

1.3 Posizionamento

Agora non compete con progetti enterprise come ContextForge (IBM), MetaMCP, AutoMem o mcp-memory-service. E' un progetto portfolio che dimostra competenza su AI infrastructure, MCP protocol, vector search, semantic routing e system design, risolvendo un problema reale nei workflow personali con agenti. Il suo valore non e' nell'originalita' del concetto, ma nella qualita' dell'esecuzione: stack 2026 (MCP, vettori, embedding), architettura pulita (7 layer, dipendenze astratte, testabile), e problematica reale (memoria isolata tra agenti).

2. Stack Tecnologico

Il progetto e' interamente sviluppato in Python 3.13+ con uv come package manager. La tabella seguente riassume i componenti principali dello stack.

|                                                                                          |                                                                                                               |                                                                                            |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
|  FastMCP |  ChromaDB (PersistentClient) |  SQLite |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|

2.1 Scelte Tecnologiche

FastMCP

L'SDK ufficiale Anthropic offre FastMCP, un wrapper ad alto livello che semplifica la creazione di server MCP. Rispetto al low-level Server, richiede meno boilerplate e permette di focalizzarsi sulla logica applicativa.

ChromaDB

Database vettoriale embedded, zero config, zero cloud. Supporta ANN index (HNSW) per ricerca in O(log n) e cosine distance. La libreria PersistentClient garantisce persistenza su disco senza server esterno.

sentence-transformers

Modello all-MiniLM-L6-v2: 384 dimensioni, ~80MB, sufficiente per MVP. Viene caricato lazy (non all'import) e cachato in ~/.cache/agora/models/.

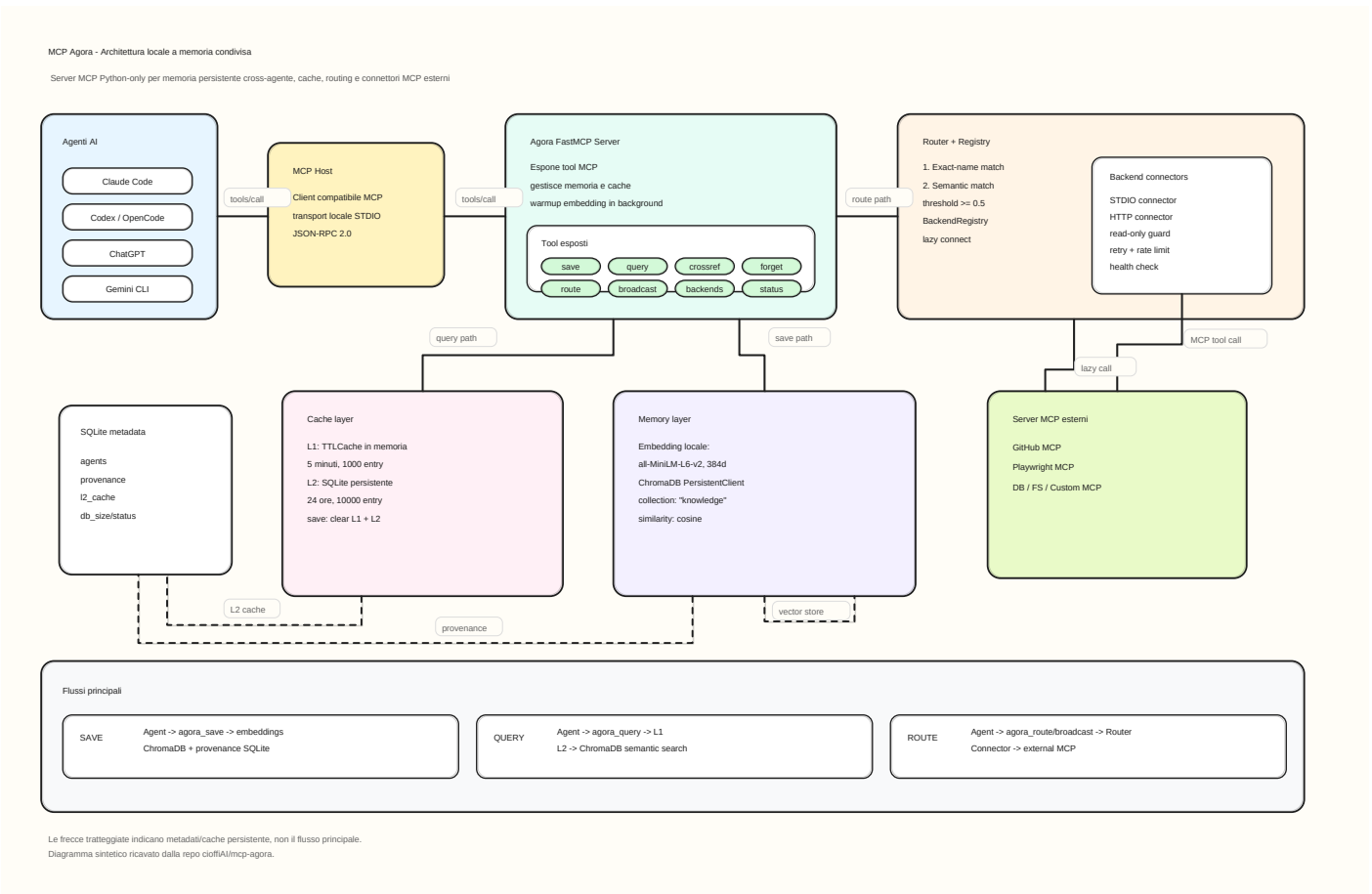
SQLite

Per metadati, provenance e cache L2. Zero configurazione, file-based, incluso nella libreria standard di Python. Ogni operazione apre e chiude una connessione (thread-safe).

3. Architettura

L'architettura di MCP Agora e' organizzata in 7 layer separati, ciascuno con responsabilita' ben definite e interfacce astratte. Questo approccio modulare permette di testare, sostituire e far evolvere ogni layer indipendentemente.

3.1 Diagramma dell'Architettura



## 3.2 Descrizione dei Layer

### Layer 1: Transport Layer

Gestisce la comunicazione con gli agenti via JSON-RPC 2.0. Supporta due trasporti: STDIO (locale, stdin/stdout, default) per agenti come Claude Code e Codex CLI; Streamable HTTP (remoto, HTTP POST + SSE) per agenti remoti o multi-sessione.

### Layer 2: Protocol Layer

Espone le primitive MCP standard: tools/list per scoprire i tool disponibili, tools/call per invocarli. Agora espone 8 tool (v. Sezione 4).

### Layer 3: Router Layer

Cuore decisionale del sistema. Quando un agente chiama `agora_route`, il router tenta prima un match esatto per nome (case-insensitive), poi un match semantico via cosine similarity sulle descrizioni embedding dei backend (soglia  $\geq 0.5$ ). Se nessun match, ritorna errore. Il routing broadcasting (`agora_broadcast`) invia la richiesta in parallelo a tutti i backend, limitato ai soli tool read-only.

### Layer 4: Memory Layer

Composto da due sotto-layer: Vector Index (ChromaDB, collezione 'knowledge', spazio coseno, 384 dimensioni) per ricerca semantica, e Relational DB (SQLite3, tabelle agents, provenance, l2\_cache) per metadati strutturati. La retention della knowledge e' permanente (finche' non cancellata con `agora_forget`). La cache L2 ha TTL configurabile (default 24h).

### Layer 5: Cache Layer

Due livelli di cache in cascata. L1: cachetools TTLCache in-memory (1000 entry, 5 minuti TTL). L2: SQLite-backed su disco (10000 entry, 24h TTL). Cache key = SHA256 della richiesta completa JSON serializzata con `sort_keys=True`. Su `agora_save` e `agora_forget`, entrambe le cache vengono invalidate (clear totale). Cascade: L1 -> L2 -> ChromaDB. Su miss ChromaDB, entrambe le cache vengono popolate.

### Layer 6: Backend Connector Layer

Ogni backend MCP esterno e' rappresentato da un connettore che implementa l'interfaccia astratta BackendConnector. Due implementazioni: StdioConnector (subprocess MCP via stdio\_client + ClientSession, lazy connect, asyncio.timeout) e HttpConnector (streamable HTTP MCP via streamablehttp\_client). La connessione e' lazy: il connettore si connette al primo uso, non all'avvio. Read-only enforcement: se un backend e' marcato read\_only, solo i tool con prefissi whitelist (list\_, get\_, fetch\_, read\_, search\_, find\_, query\_, describe\_, show\_) possono essere chiamati. I tool mutativi bloccati sollevano ReadOnlyBlockedError.

### Layer 7: Embedding Layer

Provider astratto EmbeddingProvider con implementazione concreta SentenceTransformerProvider che utilizza all-MiniLM-L6-v2 (384 dimensioni). Lazy loading: il modello viene caricato al primo uso, non all'import del modulo. Cache in ~/.cache/agora/models/. Prima chiamata ~17s (download modello + torch). Chiamate successive <1s. Il metodo warmup() permette di pre-caricare il modello all'avvio del server.

## 4. API Tools

Agora espone 8 tool MCP, registrati via decoratore @mcp.tool() su FastMCP. Tutti i tool accettano e restituiscono dizionari JSON serializzabili.

|                 |                                            |                                                                                            |                      |
|-----------------|--------------------------------------------|--------------------------------------------------------------------------------------------|----------------------|
| agora_save      | Parametri tags, agent, session, confidence | Salva agente in ChromaDB + provenance in SQLite. Registra agente.                          | Cache L1+L2          |
| agora_query     | query, top_k (=5)                          | Ricerca semantica. Cascade: L1 -> L2 -> ChromaDB.                                          | Popola L1+L2 su miss |
| agora_route     | target, tool, arguments                    | Routing a backend MCP. Exact match -> semantic (>=0.5).                                    | No                   |
| agora_broadcast | tool, arguments                            | Broadcast parallelo (asyncio.gather) a tutti i backend. Solo read-only.                    | No                   |
| agora_backends  | (none)                                     | Lista backend configurati + stato connessione.                                             | No                   |
| agora_crossref  | query, entry_id, top_k                     | Correlazioni cross-agente. Per query: raggruppa per agente. Per ID: trova entry correlate. | No                   |
| agora_forget    | entry_ids, tags, agent, dry_run            | Elimina memoria. Filtri compositi. dry_run preview. Clear L1+L2.                           | Clear L1+L2          |
| agora_status    | (none)                                     | Statistiche: entry, agenti, cache L1+L2, backend, db_size.                                 | No                   |

## 5. Schema Dati

### 5.1 ChromaDB: Vector Store

Collezione unica 'knowledge' con spazio metrico coseno. I documenti sono i testi salvati, gli embedding sono vettori a 384 dimensioni generati da all-MiniLM-L6-v2, e i metadati includono tags (formato CSV), created\_at (ISO 8601) e agent. Il wrapper VectorStore (agora/memory/vector\_store.py) astrae l'API ChromaDB e gestisce automaticamente la creazione della collezione e la serializzazione.

### 5.2 SQLite: Database Relazionale

Tre tabelle per metadati strutturati: agents (registry agenti), provenance (tracciabilita' delle entry di conoscenza), l2\_cache (cache persistente su disco).

```
CREATE TABLE IF NOT EXISTS agents (  
    id TEXT PRIMARY KEY,  
    name TEXT NOT NULL,  
    agent_type TEXT DEFAULT 'unknown',  
    first_seen TEXT NOT NULL,  
    last_seen TEXT NOT NULL,  
    sessions_count INTEGER DEFAULT 0  
);  
  
CREATE TABLE IF NOT EXISTS provenance (  
    entry_id TEXT NOT NULL,  
    source_agent TEXT NOT NULL,  
    source_session TEXT NOT NULL,  
    confidence REAL DEFAULT 0.5,  
    created_at TEXT NOT NULL,  
    PRIMARY KEY (entry_id, source_agent, source_session)  
);  
  
CREATE TABLE IF NOT EXISTS l2_cache (  
    cache_key TEXT PRIMARY KEY,  
    result_json TEXT NOT NULL,  
    ttl_seconds INTEGER NOT NULL DEFAULT 86400,  
    hit_count INTEGER DEFAULT 0,  
    created_at TEXT NOT NULL,  
    expires_at TEXT NOT NULL  
);
```

6. Testing

Il progetto adotta una strategia di test a 3 livelli: unit test (logica interna pura, nessun MCP/subprocess), integration test (stesso processo, chiamate dirette a funzioni), e MCP smoke test (via mcp.ClientSession reale su STDIO subprocess). 61 test distribuiti su 10 file, zero mock - ChromaDB e SQLite reali in directory temporanee isolate per test (fixture pytest function-scoped con temp directory). Timing non usato come metrica (flaky su Windows); si usa cache.stats()['hit\_count'].

|                           |    |                                                                       |
|---------------------------|----|-----------------------------------------------------------------------|
| File: test_integration.py | 1. | Integration test for the MCP client session, using a real subprocess. |
|---------------------------|----|-----------------------------------------------------------------------|

7. Decisioni Progettuali

Le seguenti decisioni progettuali sono state prese consapevolmente, con rationale esplicito. Non devono essere modificate senza discussione.

|                                                         |                                                                                                                   |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Decisione: Non usare mock per ChromaDB e SQLite (0.5py) | Rationale: Per garantire la robustezza e la performance, si usano database reali in directory temporanee isolate. |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|

8. Conclusioni e Roadmap

8.1 Roadmap Completata

Fase 1 - Core Memory

save/query/status con ChromaDB, sentence-transformers, TTLCache L1, FastMCP.

Fase 2 - Routing + Connectors

Router semantico + esatto, StdioConnector, HttpConnector, BackendRegistry, broadcast, read-only enforcement.

Fase 3 - Cross-Agent Memory

Provenance tracking (SQLite), L2 cache persistente, crossref, forget, parametri agent/session/confidence su save.

Fase 4 - Robustness

Logging strutturato, health check backend (healthy/unhealthy/dead), retry configurabili, rate limiting, non-blocking startup, WarmingUpError grace.

8.2 Roadmap Futura

- Fase 5: README completo, esempi reali, pubblicazione GitHub + PyPI (in corso)
- Chunking: gestione documenti multi-paragrafo con split per boundary semantico

8.3 Metriche di Successo

~~Obiettivo: implementare un agente MCP che risolva problemi reali con un'accuratezza > 80% (target).~~  
MCP Agora non e' un'invenzione. E' un esercizio di ingegneria su un problema reale (memoria isolata tra agenti) con strumenti moderni (MCP, vector search, embeddings). Se risolve il problema di 4 agenti che non condividono memoria, ha gia' vinto.

A. Appendice A: config.yaml Completo

Il file config.yaml nella directory principale del progetto controlla la configurazione del server, dello storage, della cache, dell'embedding e dei backends.

```

  agora:
    name: "Agora"
    version: "0.4.0"

  storage:
    chroma_path: "~/.agora/chroma"
    db_path: "~/.agora/agora.db"

  cache:
    l1_max_entries: 1000
    l1_ttl_seconds: 300
    l2_max_entries: 10000
    l2_ttl_seconds: 86400

  embedding:
    provider: "sentence-transformers"
    model: "all-MiniLM-L6-v2"

  backends:
    - name: "github"
      transport: "stdio"
      command: ["npx", "-y", "@modelcontextprotocol/server-github"]
      description: "GitHub API: issues, PRs, repos, code search"
      read_only: false
      timeout_seconds: 15
      max_retries: 3
      retry_delay: 1.0
      rate_limit_rps: 5
      env:
        GITHUB_TOKEN: "${GITHUB_TOKEN}"
    - name: "playwright"
      transport: "stdio"
      command: ["npx", "@playwright/mcp@latest"]
      description: "Browser automation: navigate, click, screenshot, forms"
      read_only: true
      timeout_seconds: 30
      max_retries: 2
      retry_delay: 0.5
      rate_limit_rps: 2
```

B. Appendice B: Struttura del Progetto

```

mcp-agora/
|-- pyproject.toml          # Dipendenze, build, entry point "agora"
|-- config.yaml            # Configurazione server + backend
|-- agora/
|   |-- main.py            # Entry point
|   |-- server.py          # FastMCP + 8 tools
|   |-- config.py          # YAML loader
|   |-- registry.py        # BackendRegistry
|   |-- connectors/
|   |   |-- base.py        # BackendConnector ABC
|   |   |-- stdio.py       # StdioConnector
|   |   +-- http.py        # HttpConnector
|   |-- routing/
|   |   +-- router.py      # Semantic + exact router
|   |-- embedding/
|   |   |-- base.py        # EmbeddingProvider ABC
|   |   +-- sentence.py    # sentence-transformers
|   |-- memory/
|   |   +-- vector_store.py # ChromaDB wrapper
|   |-- cache/
|   |   |-- l1_memory.py   # TTLCache
|   |   +-- l2_cache.py    # SQLite cache
|   +-- db/
|       +-- database.py    # SQLite schema + ops
|-- tests/
|   |-- test_embedding.py  # 3 tests
|   |-- test_memory.py    # 5 tests
|   |-- test_cache.py     # 5 tests
|   |-- test_l2_cache.py  # 7 tests
|   |-- test_provenance.py # 7 tests
|   |-- test_protocol.py  # 6 tests
|   |-- test_routing.py   # 8 tests
|   |-- test_connectors.py # 9 tests
|   |-- test_mcp_smoke.py  # 8 tests
|   +-- _echo_server.py
|-- docs/
|   +-- generate_pdf.py
|-- README.md
+-- LICENSE (MIT)

```

## C. Appendice C: Cronologia Commit

Cronologia dei commit sul branch master del repository GitHub.

```

14dbdc5 Initial commit: MCP Agora Phase 1 complete
e209f40 Add MIT License
005590b Add agora usage instructions to AGENTS.md
5a64014 Phase 2: Routing + Backend Connectors
12e54ae docs: update README and AGENTS.md for Phase 2
0c36648 feat: HTTP connector, read_only, broadcast, timeout
b03abbe fix: ReadOnlyBlockedError public, parallel broadcast
dab75a2 feat: Phase 3 cross-agent memory
7f01873 docs: update ARCHITECTURE.md with Phase 2/3 details, add ruff linter, generate PDF
c4aa01c feat: Phase 4 robustness - logging, health check, retry, rate limit, graceful tests
40e86f6 fix: non-blocking startup + WarmingUpError grace
90765e8 chore: add *.log and chroma.sqlite3 to .gitignore

```